

DOCUKIDS

Entrega 2 — Requerimientos, Backlog y Arquitectura

Equipo: Areco Lucas Tomas , Grano Patricio Alejandro , Bigorra Paula Antonella , Horociuk Valentina

Documento de trabajo — Scrum de equipo (4 integrantes) con apoyo de IA

0. Tablero de seguimiento

Tablero (Trello): <https://trello.com/b/VW5edIKS>

Columnas sugeridas

- Backlog
- Sprint 1
- En curso
- En revisión
- Hecho

Etiquetas del tablero

- RF (funcional)
- RNF (no funcional)
- Backend
- Frontend
- Integración
- Deuda

Nota de contexto: el equipo está compuesto por 4 integrantes, con roles distribuidos en la mayoría de las actividades (definición de requerimientos, desarrollo, revisión, documentación). El backlog se prioriza por dependencia técnica (Fase 0 → Fase 6) en lugar de por reparto fijo de roles porque muchas historias dependen de que otras estén resueltas antes (por ejemplo, no se puede respaldar en Drive sin tener primero la autorización de Google), independientemente de quién las ejecute. Esto permite que cualquier integrante disponible tome la siguiente tarea desbloqueada, en vez de esperar a que le toque "su" rol.

0.1 Distribución de roles

- **Lucas Areco** - Scrum Master | Full Stack: Responsable de liderar la metodología ágil, coordinar el flujo de trabajo del equipo y aportar al desarrollo integral de la arquitectura en frontend y backend.
- **Paula Bigorra** - Desarrolladora Frontend: Responsable de la interfaz visual, la experiencia de usuario y el diseño responsive en tonos claros y azules.
- **Patricio Grano** - Desarrollador Backend Responsable de la lógica del servidor, la seguridad, la escalabilidad y la implementación de la base de datos (modelo relacional en PostgreSQL).
- **Valentina Horociuk** - API y QA: Responsable de la integración con APIs externas, principalmente Gmail API y Drive API, y del aseguramiento de la calidad y testing.

1. Requerimientos funcionales y no funcionales

1.1 Requerimientos funcionales (RF)

ID	Requerimiento	Prioridad
RF-01	El sistema debe permitir registrar e iniciar sesión de usuarios mediante Supabase Auth.	Must
RF-02	El usuario debe poder crear, listar y eliminar perfiles de hijos (nombre, fecha de nacimiento, avatar opcional).	Must
RF-03	El sistema debe permitir subir documentos (foto o PDF) asociados a un hijo, indicando nombre y tipo.	Must

ID	Requerimiento	Prioridad
RF-04	El backend debe almacenar el archivo físico en el Google Drive del propio usuario, usando un refresh_token previamente autorizado.	Must
RF-05	El sistema debe extraer automáticamente la fecha de vencimiento/emisión del documento mediante OCR (Gemini Flash/Flash-Lite).	Must
RF-06	El sistema debe listar los documentos de un hijo mostrando su estado calculado al vuelo (vigente / por vencer / vencido).	Must
RF-07	El sistema debe permitir agendar turnos médicos y crear el evento correspondiente en Google Calendar del usuario.	Must
RF-08	El scheduler debe revisar diariamente los vencimientos y enviar un correo recordatorio 15 días antes (una sola vez por umbral cruzado).	Must
RF-09	El usuario debe poder buscar documentos por etiquetas de texto (ej.: "DNI", "pasaporte").	Should
RF-10	El usuario debe poder cambiar de hijo activo desde la cabecera, actualizando todo el contenido en tiempo real.	Should
RF-11	El usuario debe poder crear carpetas/categorías personales.	Could
RF-12	El endpoint /scheduler/check debe estar protegido por cabecera X-Cron-Secret.	Must
RF-13	La app debe mostrar un botón principal "GUARDAR" grande y visible, pensado para cuidadores no expertos.	Should
RF-14	El usuario debe poder autorizar Google (Drive/Calendar/Gmail) mediante flujo OAuth iniciado desde el backend.	Must

1.2 Requerimientos no funcionales (RNF)

ID	Requerimiento	Categoría	Prioridad
RNF-01	Todo el stack debe ser 100% gratuito (Render, Supabase, Google AI Studio, cron-job.org).	Restricción económica	Must
RNF-02	El backend no debe almacenar contenido de archivos (solo metadatos).	Privacidad	Must
RNF-03	El frontend nunca debe manipular tokens de Google (solo el backend).	Seguridad	Must
RNF-04	Las credenciales deben vivir en .env (nunca en el repo); el repo incluye solo .env.example.	Seguridad	Must
RNF-05	El sistema debe tolerar el cold start de Render (30–60 s) sin romper la UX (feedback de carga).	Disponibilidad	Must
RNF-06	El cron diario debe ejecutarse aun si el servidor está dormido (cron externo, no APScheduler).	Confiabilidad	Must
RNF-07	El JWT emitido por Supabase debe ser validado por el backend en cada endpoint protegido.	Seguridad	Must
RNF-08	La URL de autorización Google debe incluir access_type=offline&prompt=consent para asegurar refresh_token.	Correctitud OAuth	Must
RNF-09	El cron diario actúa como keep-alive que evita la pausa de Supabase (7 días sin actividad).	Mantenibilidad	Must
RNF-10	Uso de modelos Gemini Flash / Flash-Lite (Pro queda prohibido por costo).	Restricción económica	Must
RNF-11	La UI debe ser usable por cualquier padre, madre o tutor sin conocimientos técnicos (botón grande, textos claros, sin menús ocultos).	Usabilidad	Should
RNF-12	La app debe funcionar en Android (APK de release generado localmente con expo run:android --variant release).	Portabilidad	Must

ID	Requerimiento	Categoría	Prioridad
RNF-13	Se documenta como limitación conocida: en el tier gratuito de Gemini, Google puede usar el contenido para mejorar sus modelos.	Privacidad (aceptada)	Should
RNF-14	El código debe estar en TypeScript (frontend) y Python 3.10+ (backend), con estructura de carpetas fija.	Mantenibilidad	Should

2. Historias de usuario priorizadas con criterios de aceptación

Formato: Como <rol> quiero <acción> para <beneficio> + criterios en formato Gherkin (Dado / Cuando / Entonces). Una historia no se puede dar por terminada sin sus criterios de aceptación cumplidos.

Épica A — Autenticación y acceso

HU-01 · Registro e inicio de sesión (Must)

Como padre/madre, quiero registrarme e iniciar sesión con email y contraseña, para acceder a mi espacio privado de documentos.

Criterios de aceptación:

- Dado que estoy en la pantalla de login, cuando ingreso email y contraseña válidos contra Supabase Auth, entonces recibo un JWT y accedo a la pantalla principal de Docs.
- Dado que ingreso credenciales inválidas, cuando confirmo, entonces veo un mensaje de error claro sin exponer detalles técnicos.
- Dado que ya estoy logueado, cuando cierro y reabro la app, entonces mantengo la sesión activa (persistencia de JWT).

HU-02 · Autorización con Google (Must)

Como padre/madre, quiero conectar mi cuenta de Google una sola vez, para que Docukids pueda guardar mis archivos y crear turnos en mi nombre.

Criterios de aceptación:

- Dado que estoy en Ajustes, cuando toco "Conectar con Google", entonces se abre el navegador con la URL generada por /auth/google/url que incluye access_type=offline&prompt=consent.
- Dado que autorizo la app en Google, cuando Google redirige a /auth/google/callback, entonces el backend guarda el refresh_token asociado a mi usuario y redirige al frontend con ?google=ok.
- Dado que ya autorizé antes, cuando repito el flujo, entonces obtengo igualmente un refresh_token válido (no se rompe en silencio).

Épica B — Gestión de hijos

HU-03 · Crear perfil de hijo (Must)

Como padre/madre, quiero crear el perfil de mi hijo, para asociarle sus documentos y turnos.

Criterios de aceptación:

- Dado que estoy en la pestaña "Hijos", cuando completo name y birth_date y confirmo, entonces el backend crea el registro vía POST /children y aparece en la lista.
- Dado que intento crear un hijo sin nombre, cuando confirmo, entonces recibo validación 422 y no se crea nada.
- Dado que creé un hijo, cuando vuelvo a Docs, entonces ese hijo aparece disponible en el selector de cabecera.

HU-04 · Cambiar de hijo activo (Should)

Como padre/madre, quiero cambiar el hijo activo desde la cabecera, para ver solo su documentación.

Criterios de aceptación:

- Dado que tengo 2 hijos cargados, cuando toco la flecha ▼ de la cabecera y elijo otro, entonces la lista de documentos, calendario y alertas cambian a ese hijo en tiempo real.
- Dado que cambio de hijo, cuando la vista se actualiza, entonces el nombre del hijo activo se refleja en la cabecera y en la pestaña Calendario.

Épica C — Subida y almacenamiento de documentos

HU-05 · Subir documento (Must)

Como padre/madre, quiero sacar una foto o elegir un PDF y guardarlo asociado a un hijo, para no perderlo.

Criterios de aceptación:

- Dado que estoy en Docs, cuando toco el botón "GUARDAR", entonces se abre la cámara o galería (expo-image-picker).
- Dado que elegí un archivo, cuando completo name (ej. "DNI") y opcionalmente type (ej. "identidad"), entonces se envía multipart/form-data a POST /upload con file, child_id, name, type.
- Dado que la subida fue exitosa, cuando recibo la respuesta, entonces obtengo { status, expiry_date, drive_link, doc_id } y el documento aparece en la lista.
- Dado que la subida falla por cold start, cuando espero entre 30–60 s, entonces la app muestra un indicador de carga y reintenta sin perder el archivo.

HU-06 · Respaldo invisible en Google Drive (Must)

Como padre/madre, quiero que mis archivos queden en la nube sin configurar nada, para no depender de la memoria del teléfono.

Criterios de aceptación:

- Dado que subo un documento, cuando el backend lo procesa, entonces usa mi refresh_token para subirlo a mi Drive y guarda drive_file_id + drive_link en la tabla documents.
- Dado que el archivo está en Drive, cuando consulto GET /documents/{child_id}, entonces recibo el drive_link para verlo/descargarlo.
- Dado que el backend se rompe, cuando audito el servidor, entonces confirmo que no hay copia del archivo físico en el backend (solo metadatos).

Épica D — OCR y alertas automáticas

HU-07 · Lectura automática de fecha (OCR) (Must)

Como padre/madre, quiero que la app lea la fecha de vencimiento del documento, para no tener que tipearla.

Criterios de aceptación:

- Dado que subí un documento con fecha visible, cuando el backend llama a Gemini (gemini-2.0-flash-lite o gemini-flash), entonces recibo la fecha de vencimiento y la guardo en expiry_date.
- Dado que el OCR no puede leer una fecha, cuando responde, entonces el sistema ofrece carga manual de fecha (fallback) sin bloquear la subida.
- Dado que el modelo pedido es Pro, cuando se ejecuta la llamada, entonces el sistema lo rechaza por configuración (Pro está prohibido por costo).

HU-08 · Estado visual del documento (Must)

Como padre/madre, quiero ver con colores qué documento está por vencer, para detectar urgencias de un vistazo.

Criterios de aceptación:

- Dado que un documento vence en ≤ 30 días, cuando veo la lista, entonces aparece en naranja ("próximo a vencer").
- Dado que un documento tiene `expiry_date < hoy`, cuando veo la lista, entonces aparece en rojo ("vencido").
- Dado que un documento tiene `expiry_date` lejano, cuando veo la lista, entonces aparece en verde ("vigente").
- Dado que cambio la fecha del sistema, cuando recargo la lista, entonces el status se recalcula (no está persistido en la DB).

HU-09 · Recordatorio por correo — función más importante (Must)

Como padre/madre, quiero recibir un correo 15 días antes del vencimiento, para no olvidarme de renovar.

Criterios de aceptación:

- Dado que hoy es 9/9 y un documento vence el 10/10, cuando corre el cron diario, entonces calcula "faltan 31 días" y no dispara nada.
- Dado que un documento cruza el umbral de 15 días, cuando corre el cron, entonces envía un solo correo a la dirección del usuario con el asunto "¡El DNI de tu hijo vence en 15 días!".
- Dado que ya se envió el correo de ese umbral, cuando corre el cron al día siguiente, entonces no vuelve a enviarlo (idempotencia).
- Dado que nadie abrió la app, cuando corre el cron externo, entonces el correo igual llega (no depende de la app abierta).

Épica E — Turnos y calendario

HU-10 · Agendar turno médico (Must)

Como padre/madre, quiero agendar un turno desde la app, para verlo junto a mis otras citas en Google Calendar.

Criterios de aceptación:

- Dado que estoy en la pestaña Calendario, cuando toco "+" y completo `child_id`, `title`, `date`, `time`, `notes?`, entonces el backend crea el evento en Google Calendar del usuario.
- Dado que el evento se creó, cuando recibo la respuesta, entonces obtengo `{ ok, calendar_event_id }` y guardo el `calendar_event_id` en la tabla `appointments`.
- Dado que tengo Google Calendar instalado, cuando abro la app de Google, entonces veo la cita junto al resto de mis eventos.
- Dado que el sistema escribe en Google Calendar, cuando reviso el calendario nativo de iOS/Android, entonces no aparece ahí (por diseño, fuera del MVP).

HU-11 · Vista de calendario en la app (Should)

Como padre/madre, quiero ver un calendario dentro de Docukids con mis turnos, para no salir de la app.

Criterios de aceptación:

- Dado que estoy en la pestaña Calendario, cuando se carga la vista, entonces veo el mes actual con el día de hoy resaltado (react-native-calendars, solo vista).
- Dado que hay turnos cargados, cuando miro debajo del calendario, entonces veo la lista (`GET /appointments/{child_id}`).

Épica F — Búsqueda y usabilidad

HU-12 · Búsqueda por etiquetas (Should)

Como abuelo/cuidador, quiero escribir "pasaporte" y encontrar el documento al instante, para ahorrar tiempo buscando manualmente.

Criterios de aceptación:

- Dado que estoy en Docs, cuando escribo en la barra de búsqueda, entonces la lista se filtra en tiempo real por coincidencia en name o type.
- Dado que la búsqueda no arroja resultados, cuando termino de escribir, entonces veo un estado vacío claro ("No se encontraron documentos").
- Dado que la búsqueda es por texto escrito, cuando reviso la implementación, entonces confirmo que no hay búsqueda por voz (limitación de recursos del proyecto).

HU-13 · Botón "GUARDAR" para no expertos (Should)

Como abuelo/cuidador, quiero un botón enorme y visible para guardar, para no perderme en menús.

Criterios de aceptación:

- Dado que abro Docs, cuando miro la pantalla, entonces el botón "GUARDAR" es visualmente dominante (área táctil amplia, contraste alto).
- Dado que toco "GUARDAR", cuando se abre la cámara/galería, entonces el flujo es de un solo paso hasta la subida.

Épica G — Privacidad y limitaciones

HU-14 · Panel de Ajustes con estado de conexión (Should)

Como padre/madre, quiero ver en Ajustes si Google está conectado, para saber si mis archivos se están respaldando.

Criterios de aceptación:

- Dado que no autoricé Google, cuando abro Ajustes, entonces veo "Google: no conectado" + botón "Conectar con Google".
- Dado que sí autoricé, cuando abro Ajustes, entonces veo "Google: conectado" + opción de reconectar.

HU-15 · Aviso explícito de privacidad de Gemini (Should)

Como responsable del proyecto, quiero documentar la limitación de privacidad del tier gratuito de Gemini, para que el informe académico la refleje.

Criterios de aceptación:

- Dado que el sistema usa Gemini free tier, cuando se revisa el informe, entonces aparece la nota: "El nivel gratuito puede usar el contenido para mejorar modelos de Google; se acepta por alcance académico".
- Dado que existe esta limitación, cuando un usuario sube un documento, entonces se lo informa en la pantalla de Ajustes o en un aviso la primera vez.

3. Estimación de las historias del primer sprint

Duración del sprint: 2 semanas (Sprint 1). Unidad: puntos de historia (Fibonacci: 1, 2, 3, 5, 8, 13). Capacidad estimada: equipo de 4 integrantes con roles distribuidos ≈ 32–38 puntos por sprint, apoyado por herramientas de IA para acelerar tareas puntuales (boilerplate, debugging, documentación).

3.1 Estimación inicial (candidatas a Sprint 1)

Orden	HU	Puntos	Justificación breve
1	HU-01 Registro/login (Supabase directo)	3	SDK resuelto; solo interceptor Axios + persistencia
2	HU-02 Autorización Google OAuth	8	Riesgo alto: refresh_token, access_type=offline, callback
3	HU-03 Crear perfil de hijo	2	CRUD simple
4	HU-04 Cambiar hijo activo	3	Estado global + re-fetch
5	HU-05 Subir documento	5	Multipart + integración cámara
6	HU-06 Respaldo en Drive (backend)	8	Depende de HU-02; subida real a Drive
7	HU-07 OCR con Gemini	5	Modelo Flash + fallback manual
8	HU-08 Estado visual por colores	3	Cálculo al vuelo en backend + UI
9	HU-12 Búsqueda por etiquetas	3	Filtro local
10	HU-13 Botón GUARDAR grande	2	Solo UI

Total inicial: 42 puntos ⚠ Excede la capacidad estimada del sprint.

3.2 Ajuste del Sprint 1 (recorte realista)

HU	Puntos	Estado en Sprint 1
HU-01 Registro/login	3	✓ Incluida
HU-02 OAuth Google	8	✓ Incluida (crítica para todo lo demás)
HU-03 Crear hijo	2	✓ Incluida
HU-04 Cambiar hijo	3	✓ Incluida
HU-05 Subir documento	5	✓ Incluida
HU-06 Respaldo Drive	8	✓ Incluida (núcleo)
HU-07 OCR Gemini	5	✓ Incluida
HU-08 Estado colores	3	⏮ Mover a Sprint 2
HU-12 Búsqueda	3	⏮ Mover a Sprint 2
HU-13 Botón GUARDAR	2	⏮ Mover a Sprint 2

3.3 Alcance final del Sprint 1

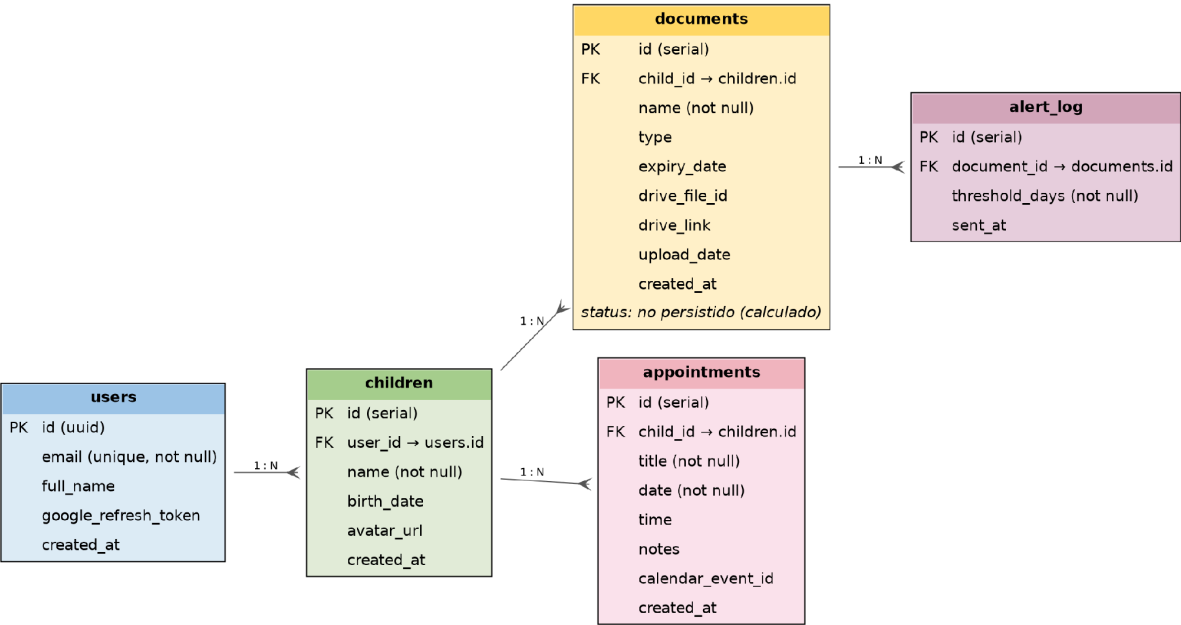
HU-01 (3) + HU-02 (8) + HU-03 (2) + HU-04 (3) + HU-05 (5) + HU-06 (8) + HU-07 (5) = 34 puntos

Nota: El recorte a 34 pts responde a un criterio conservador de primer sprint (curva de aprendizaje del equipo con las integraciones externas —OAuth, Drive, Gemini—), no a una limitación de capacidad del equipo.

Sprint 2 (estimado): HU-08, HU-09, HU-10, HU-11, HU-12, HU-13, HU-14, HU-15.

4. Modelo de datos (diagrama de entidades)

A continuación, el diagrama entidad-relación y el detalle de cada tabla.



4.1 Detalle de tablas

users

Campo	Tipo	Notas
id	uuid	PK
email	text	unique, not null
full_name	text	
google_refresh_token	text	
created_at	timestamp	default now()

children

Campo	Tipo	Notas
id	serial	PK
user_id	uuid	FK → users.id, not null
name	text	not null
birth_date	date	
avatar_url	text	
created_at	timestamp	default now()

documents

Campo	Tipo	Notas
id	serial	PK
child_id	int	FK → children.id, not null
name	text	not null
type	text	
expiry_date	date	
drive_file_id	text	
drive_link	text	
upload_date	date	default current_date
created_at	timestamp	default now()

status no se persiste; se calcula al vuelo comparando expiry_date con hoy.

appointments

Campo	Tipo	Notas
id	serial	PK
child_id	int	FK → children.id, not null
title	text	not null
date	date	not null
time	time	
notes	text	
calendar_event_id	text	
created_at	timestamp	default now()

alert_log

Campo	Tipo	Notas
id	serial	PK
document_id	int	FK → documents.id
threshold_days	int	not null
sent_at	timestamp	default now()

Tabla auxiliar para idempotencia del correo (RF-08). Evita reenviar el mismo aviso.

4.2 Relaciones

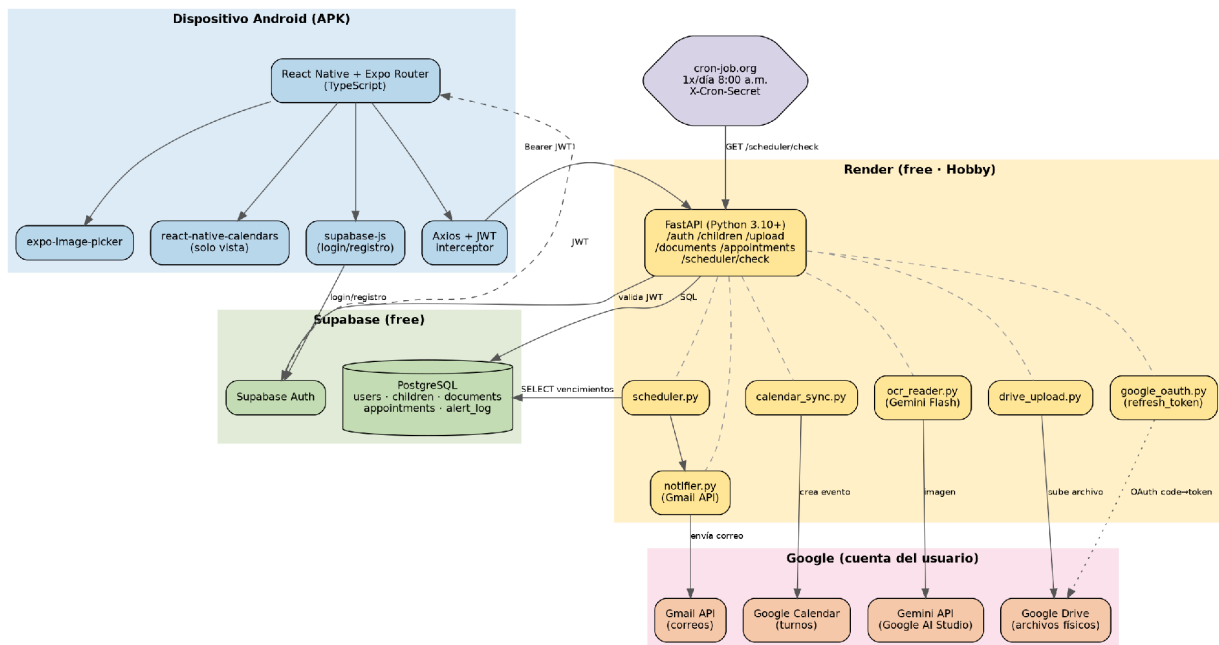
- users 1 — N children (ON DELETE CASCADE)
- children 1 — N documents (ON DELETE CASCADE)
- children 1 — N appointments (ON DELETE CASCADE)
- documents 1 — N alert_log (control de envíos únicos)

4.3 Decisiones clave del modelo

- status NO se persiste — se calcula al vuelo (vencido / por_vencer / vigente) comparando expiry_date con hoy. Evita inconsistencias por paso del tiempo.
- No se guardan archivos — solo drive_file_id + drive_link. Los PDFs/fotos viven en el Drive del usuario.
- alert_log — añadida para cumplir el RNF de idempotencia: el email solo se dispara una vez al cruzar el umbral.

5. Diagrama de arquitectura

Herramienta sugerida para el diagrama editable: Mermaid / draw.io.



Narrativa del diagrama (qué habla con qué)

- App móvil → Supabase Auth: registro/login directo desde el frontend (no hay /auth/register ni /auth/login en FastAPI). El frontend guarda el JWT.
- App móvil → FastAPI: todas las llamadas van con Authorization: Bearer <JWT>; el backend valida el JWT contra Supabase.
- FastAPI → Supabase Postgres: lee/escribe metadatos (nunca archivos).
- FastAPI → Google Drive: sube el archivo usando el refresh_token del usuario.
- FastAPI → Gemini: envía la imagen para OCR (modelo Flash). Único punto donde el contenido sale brevemente del control del backend.
- FastAPI → Google Calendar: crea el evento del turno.
- FastAPI → Gmail: envía el correo recordatorio.
- cron-job.org → FastAPI: dispara /scheduler/check 1x/día con X-Cron-Secret. Este ping cumple doble función: (a) revisa vencimientos, (b) evita que Supabase pause el proyecto por 7 días de inactividad (RNF-09).
- Frontend nunca toca tokens de Google (RNF-03).

Nota sobre cold start: Render duerme el backend a los 15 min sin uso. El cron diario lo despierta; la primera petición puede tardar 30–60 s.

6. Definition of Done (DoD) del equipo

Una historia se considera terminada cuando cumple todos estos criterios. Se respeta sin excepciones hasta noviembre.

6.1 Criterios generales

- Código mergeado en la rama principal (o carpeta compartida) sin conflictos.
- Compila y levanta sin errores localmente: `uvicorn app.main:app --reload` (backend) y `npx expo start` (frontend).
- Los criterios de aceptación de la HU se cumplen y se probaron manualmente (no basta con "anda en mi cabeza").
- No rompe funcionalidad previa: al menos un smoke test de las HUs anteriores sigue pasando.
- No quedan `console.log` de debug, `print()` de debug ni TODOs sin ticket asociado.
- `.env` no está commiteado. Se actualizó `.env.example` si se agregó una variable nueva.
- No se introdujeron dependencias de pago (verificadas contra la tabla de servicios gratuitos).
- No se usa `gemini-pro` ni ningún modelo Gemini de pago (RNF-10).
- El frontend no manipula tokens de Google (RNF-03).
- El backend no guarda contenido de archivos (RNF-02).
- La historia fue revisada por al menos otro integrante del equipo antes de pasar a "Hecho" (revisión funcional o de código).

6.2 Criterios específicos por tipo

Tipo	Criterio adicional
Endpoint nuevo	Documentado automáticamente en /docs (FastAPI) y probado con Postman/curl. Respuesta de error documentada (4xx/5xx).
Integración externa	Probada una por una contra el servicio real (Drive, Calendar, Gmail, Gemini), no solo mockeada.
Pantalla nueva (frontend)	Respetando el mockup correspondiente. Probada en emulador Android y en dispositivo físico.
Scheduler / cron	Verificado manualmente que el correo llegue antes de confiar en el cron.
OAuth	Verificado que llega <code>refresh_token</code> incluso en segunda autorización (con <code>access_type=offline&prompt=consent</code>).

6.3 Criterios de cierre del Sprint

- Backend desplegado en Render con variables de entorno cargadas en el panel (no desde `.env`).
- Cron apuntando a la URL pública de Render (no a localhost).
- Commit/zip de respaldo creado: `version_estable_X.X.zip` o tag de Git.
- Demo funcional de punta a punta de al menos el núcleo: Drive + OCR + Google Calendar + correo automático diario.

6.4 Qué NO hace falta para dar por terminada una HU

- No hace falta cobertura de tests unitarios (no es requisito de la cátedra).
- No hace falta documentación extensa por HU (basta con el informe final y este documento).
- No hace falta que esté en producción para el usuario final (alcanza con funcionar en el entorno de demo).